AIGC 5500 | MODULE 05 | 4% OF FINAL GRADE

Lab 05: Attention Against Recurrence

Due date posted on Blackboard

Module 05 made four claims about attention: that a learned lookup removes the distance problem of recurrence, that multi-head attention is a reshape and not extra parameters, that attention cannot see order unless it is given positions, and that the layout of normalization decides whether a deep stack trains at all. In this lab you build attention yourself, check it against PyTorch, and then test each claim on a small task with a known answer. What is being assessed is whether your implementation is correct, whether your comparisons are controlled, and whether your conclusions stay inside what your measurements show.

Starter code

You are given lab05_starter.py. It contains the delayed-recall task from Module 04, a new order task, an attention readout model, an LSTM recall model, a small Transformer classifier for the order task, and training loops that accept your attention code as an argument. Until you write your own, the starter uses PyTorch's attention, so every harness runs on day one. You write the attention, the multi-head layer, the four experiments, the timing and the conclusion. Do not edit the starter file directly; copy it into your repository and import from it, or copy what you need into your own work.

Two tasks. In delayed recall, step 0 carries a class label (4 classes) and every later step is noise; the model reports the label, and chance is 0.25. In the order task, every sequence contains token 1 once and token 2 once plus filler, and the label says which comes first; both classes contain the same tokens, so only order separates them, and chance is 0.50.

```
from lab05_starter import train_recall, train_order, reference_attention, ReferenceMHA

# Delayed recall: kind is "attention" or "lstm". Returns (accuracy, weight on step 0).
accuracy, weight0 = train_recall("attention", seq_len=100, seed=0,
attention_fn=my_attention)
accuracy, _ = train_recall("lstm", seq_len=100, seed=0)

# Order task: returns one accuracy per length in test_lens.
accs = train_order("sinusoidal", seed=0, test_lens=(16, 32), attention_cls=MyMHA)

# Deep stack with layout and warmup options.
accs = train_order("sinusoidal", seed=0, depth=12, norm="post", warmup=100,
                   steps=400, lr=3e-3, test_lens=(16,), attention_cls=MyMHA)
```

Run lab05_starter.py once, unmodified, to confirm your environment works before changing anything. It should report accuracies near 1.0 for both tasks and finish in under a minute.

What to build

Four pieces, in order.

1. **Attention from scratch, verified.** Write scaled_dot_product_attention(q, k, v, mask=None) returning (output, weights), and a MultiHeadAttention(d_model, n_heads) module with the call signature module(x, context=None, mask=None). Use only tensor operations and nn.Linear. Show with code that your function matches PyTorch's on random inputs with and without a causal mask, and that your module matches nn.MultiheadAttention after you copy its weights across, for self-attention, causal self-attention and cross-attention (context with a different length). Print the parameter count for 1, 2, 4 and 8 heads at d_model 32 and explain the result in one sentence. Show that shuffling the input of a layer shuffles its output in the same way.
2. **Recall: attention against an LSTM.** At lengths 30, 100 and 400, train the attention readout (with your own attention) and an LSTM with forget bias 3, three seeds each (0, 1 and 2), at the starter's defaults. Report every seed, plus the

attention weight on step 0 for the attention model. Then explain, using the weights and the structure of each model, why distance matters to one and not the other, and say exactly what this experiment does not show.

3. **Positions on the order task.** Train the order classifier with no positions, learned positions and sinusoidal positions on length 16, and test at lengths 16 and 32, three seeds each, using your multi-head layer. Write a prediction in your README before running the next step: if training uses lengths drawn at random from 8, 16 and 24 (mixed_lens), will accuracy at length 32 improve for the learned and for the sinusoidal version, and why? Then run it, testing at 16, 32 and 48, three seeds each, and compare with your prediction.
4. **Normalization and cost.** Train 12-block stacks on the order task for 400 updates at learning rate 3e-3, three seeds each, in three settings: post-norm with no warmup, post-norm with a 100-update warmup, and pre-norm with no warmup. Then time a forward and backward pass of an LSTM and of one attention layer of matching width at sequence lengths 100, 200, 400 and 800 (batch 32, best of three repeats). Write a conclusion that says what attention removed, what it costs, and what your own numbers do and do not establish.

Requirements

Your submission must do all of the following. Each is checked directly during marking.

- Write the attention function and the multi-head module yourself. Calling torch.nn.functional.scaled_dot_product_attention or nn.MultiheadAttention inside your own implementation does not count; they are there to check against.
- Apply the mask before the softmax, take the softmax over the keys axis, and divide by the square root of the key dimension. Assert in your code that each row of weights sums to 1 and that masked positions are exactly 0.
- Compare configurations under identical conditions: the same data generator, the same validation sets, the same updates, optimizer and learning rate. The only things that may differ between two compared runs are the variable being tested.
- Report every seed, not only the mean. Three seeds per cell throughout.
- Write your mixed-length prediction in your README before running it, and keep it in the final README even if it turned out to be wrong.
- Quote accuracy relative to chance (0.25 for recall, 0.50 for the order task). Use the measurements to argue each claim; do not argue from what a diagram or the module suggests.
- Report surprising or inconvenient results honestly, such as a seed that fails, an encoding that does not extrapolate, or a stack that trains without warmup. A null result explained correctly is worth more than a flattering table.
- State the limits of your evidence: one small task, a fixed update budget, a handful of seeds, and timings that depend on your machine.

Watch out

Three silent mistakes run without errors and still give plausible numbers. The softmax over the wrong axis. A mask applied after the softmax, which leaves the rows summing to less than 1. A reshape that splits heads in the wrong order, which mixes values from different positions. Your comparison against PyTorch is what catches these, so write it first and run it on a mask and on a context of a different length, not only on the easy case.

Why a prediction, and why report the failures

A prediction written before the run is the only way to tell whether you understood the mechanism or only explained the result afterwards. Positional encodings are the sharpest case in this lab: the module explains why a sinusoidal form could extrapolate and why a learned table cannot, and your measurement decides whether it does. Whatever you find, report it, and explain it with the mechanism, not with the name of the encoding.

Repository requirements

The repository is as much a part of the submission as the code. It must contain the following, and it must not contain your virtual environment.

Item	Requirement
.gitignore	Present, committed before your first code commit, and excluding .venv, __pycache__, .env, and any large output files
README.md	Explains the task, each of the four pieces, your mixed-length prediction, your results tables, and how to reproduce your results
requirements.txt	Generated with pip freeze, including torch and matplotlib
Your code	Your attention and multi-head module, the verification against PyTorch, the experiments and the timing, as scripts or a single notebook, together with lab05_starter.py
Figures	At least one figure saved as an image and referenced from the README, for example the recall results against sequence length or the timing against length
Commit history	At least three commits, each a coherent change, with messages that describe what changed

README template

You may use this structure directly. Replace the content, not the shape.

```
# Attention Against Recurrence

Attention built from scratch, then tested against recurrence, positions
and normalization on small tasks with known answers.

## Setup
python -m venv .venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate
pip install -r requirements.txt

## Run
python attention.py     # piece 1: my attention, checks against PyTorch
python recall.py        # piece 2: attention readout against LSTM
python positions.py     # piece 3: none, learned, sinusoidal
python depth_and_cost.py  # piece 4: 12-block stacks and timing

## Prediction (written before running the mixed-length training)
What I expected for learned and sinusoidal positions at length 32, and why.

## Results
Verification: what matched PyTorch, and the parameter counts.
Recall: accuracy per seed at each length, and the weight on step 0.
Positions: accuracy per seed at each length, before and after mixing lengths.
Stacks and timing: accuracy per seed, and the timing table.

## Conclusion
What attention removed, what it costs, and what these measurements
do and do not establish.

## Known limitations
Anything you are aware of that does not work, or that you would improve
with more time.
```

How to submit

Post the URL of your GitHub repository to the Lab 05 submission on Blackboard. Do not upload files, and do not submit a link to a single file within the repository.

```
https://github.com/your-username/your-repository
```

Make sure the repository is public. If you prefer to keep it private, you must share it with the instructor account below. Sharing with any other account does not count as a submission and will be rejected.

Instructor GitHub account

`hossein.pourmodheji@humber.ca`

Add this account as a collaborator under Settings, Collaborators in your repository. Any other account will be rejected.

A repository that a marker cannot open is treated as not submitted, and the late policy applies from the point at which access is granted.

Before you submit, run this check

Clone your own repository into a different folder, create a fresh virtual environment, install from requirements.txt, and reproduce every table and figure following only your README. The full set of runs takes roughly fifteen minutes on a laptop CPU, mostly the LSTM at length 400 and the 12-block stacks. If everything completes cleanly and agrees with what you reported, your submission is complete.